

RL

Notes taker: Alex

Contact: wang-zx23@mails.tsinghua.edu.cn

Reference: Mingsheng Long ML Lecture9-10 Huazhe Xu DRL Lecture 1-2

RL

0 Intro

1 MDP

1.1 Basics

1.2 Markov Process

1.3 Markov Decision Process

2 Model-based Learning

Policy Iteration

Value Iteration

3 Model-free learning

MC-Value Iteration

TD

TD(0)

TD(λ)

SARSA

SARSA

SARSA(λ)

Q-learning

0 Intro

- 强化学习广泛应用于序列决策问题中，是机器学习的一个子领域，与监督学习、无监督学习都有交集。对于每一个time step t ，智能体接受一个观察 O_t ，得到一个标量监督信号**scalar cumulative reward** R_t ，实施一个动作**action** A_t ，环境**environment**随时间迭代。
- 强化学习中的**history**视为 O_t, R_t, A_t 的序列，**state**是**history**的函数， $S_t = f(H_t)$
- 奖励 R_t 是一个标量信号，强化学习的基本假设是**Reward Hypothesis**，即一个目标可以被最大化积累奖励所刻画。
- Agent会对世界模型的观察可以分为完全观察和部分观察，完全观察对应的是 $O = S^a = S^e$ 这时对应的是MDP，马尔科夫决策过程；然而实际上在实际的任务中，智能体自身的状态是环境状态的一个子集，观察只能是部分的 $S^a \neq S^b$ ，这时对应的是POMDP。

1 MDP

1.1 Basics

RL的基本要素：Objective, State, Action, Reward

对于一个follow RL算法的agent，用策略函数 π 来刻画agent的行为，agent采取的策略可以分为 $a = \pi(s)$ 决定性策略和随机性策略 $\pi(a|s) = \mathbb{P}(A_t = a|S_t = s)$ ；使用价值函数来评估一个状态/策略的好坏：一般约定使用 $V_\pi(s)$ 和 $Q_\pi(s, a)$ 。RL中的**model**分为对环境的建模和对奖励函数的建模，通过转移概率和期望的形式反映未来状态和奖励与当前状态、动作以及历史的关系。一般约定 $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s'|S_t = s, A_t = a]$ $\mathcal{R}_s^a = \mathbb{E}[R_{t+1}|S_t = s, A_t = a]$ 。考虑到奖励信号常是延迟的，我们需要对预期奖励进行建模。

eg:在机器人控制领域，Objective是使得robot move forward，state是joints的angle和position，action是joints上的torques，reward是专门设计出用于激励机器人不跌倒、向前移动的奖励

1.2 Markov Process

- **Markov Process Definition:** $P(S_{t+1}|S_t) = P(S_{t+1}|S_1, \dots, S_t)$, 则称 S_t 是Markov的。一个MP由 $\langle S, P \rangle$ 的tuple构成。S是所有可能状态的集合，P是状态转移矩阵。
- **Markov Reward Process:** 在以上的MP中，加入对于reward function $\mathcal{R}_s = \mathbb{E}[R_{t+1}|S_t = s]$ 和用于计算累计奖励的折扣因子 $\gamma \in [0, 1]$ ，在累计奖励计算中，可以使用总累计回报 $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ ，折扣因子体现了考虑到未来的不确定性对于决策的影响，折扣因子越大，体现的是比较far-sighted的评估，而近似于0的则比较myopic。瞬时奖励 G_t 是r.v.
- **Value function:** 从状态s出发，MRP的价值函数 $v(s)$ 刻画的是期望回报。 $V(s) = \mathbb{E}[G_t|S_t = s]$.
- **Bellman Equation:** 可以将 G_t 分为当前的奖励和下一步的奖励，有 $V(s) = \mathbb{E}[R_{t+1} + \gamma V(S_{t+1})|S_t = s] = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} P_{ss'} V(s')$

proof:

$$\begin{aligned} V(s) &= \mathbb{E}[G_t|S_t = s] \\ &= \mathbb{E}[R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots |S_t = s] \\ &= \mathbb{E}[R_t + \gamma(R_{t+1} + \gamma R_{t+2} + \dots) |S_t = s] \\ &= \mathbb{E}[R_t + \gamma G_{t+1} |S_t = s] \\ &= \mathbb{E}[R_t + \gamma V(S_{t+1}) |S_t = s] \end{aligned}$$

这里解释最后一个（笔者认为并不显然的）等式：

利用重期望法则（条件期望的重期望法则）： $E[X|Z = z] = E[E[X|Y]|Z = z]$ 同时我们还有 $V(s) = \mathbb{E}[G_t|S_t = s]$ ，即 $V(S_{t+1}) = \mathbb{E}[G_{t+1}|S_{t+1}]$ 那么有 $E[V(S_{t+1})] = \mathbb{E}[G_{t+1}|S_{t+1}]|S_t = s] = \mathbb{E}[G_{t+1}|S_t = s]$.

写成矩阵代数的形式： $\mathbf{v} = \mathbf{R} + \gamma \mathbf{P}\mathbf{v}$ 这个线性方程其实是有解的。

1.3 Markov Decision Process

- **Definition:** 常用上标 $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s'|S_t = s, A_t = a]$ $\mathcal{R}_s^a = \mathbb{E}[R_{t+1}|S_t = s, A_t = a]$ 来表示动作a。而agent follow的policy可以定义为在状态s上动作a的distribution，即 $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$ ，Markov Decision Process中的agent的policy只取决于当前的状态，而与历史状态无关，同时其形式与时间也是无关的（平稳性，时间上的参数共享）： $A_t \sim \pi(\cdot|S_t), \forall t > 0$
- **State Value function:** (状态价值函数) $V^\pi(s) = \mathbb{E}^\pi[G_t|S_t = s]$
- **Action Value function:** (动作价值函数) $Q^\pi(s, a) = \mathbb{E}^\pi[G_t|S_t = s, A_t = a]$
- **Bellman Equation:** 以下三组等式可以相互推导，都是贝尔曼期望方程。本质上其实是全概率公式的展开。

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1})|S_t = s] \\ Q^\pi(s, a) &= \mathbb{E}_\pi[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1})|S_t = s, A_t = a] \\ V^\pi(s) &= \sum_{a \in \mathcal{A}} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V^\pi(s') \right) \\ Q^\pi(s, a) &= R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') Q^\pi(s', a') \end{aligned}$$

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) Q^\pi(s, a)$$

$$Q^\pi(s, a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V^\pi(s')$$

- **Optimal Policy:** 一般用 $V^\pi(s) \forall s$ 在 policy 上建立偏序关系, $\pi \geq \pi'$ if $V^\pi(s) \geq V^{\pi'}(s) \forall s$, 那么在 MDP 中, 可以保证最优策略 π^* 的存在性, 且在最优策略下, 每一个状态 s 可以获得最大的状态价值函数, 每一个状态动作对也可以获得最大的动作价值函数。如果知道了最优的动作价值函数 $Q^*(s, a')$ 我们可以直接获得最优策略 (由动态规划的最优子结构的存在性保证)
 $\pi^* = \mathbb{1}[\operatorname{argmax}_{a \in \mathcal{A}} Q^*(s', a)]$ 但是找到 Q^* 是难的。
- **Bellman Optical Equation:**
 价值迭代使用的就是贝尔曼最优方程。

$$V^*(s) = \max_{a \in \mathcal{A}} \{r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V^*(s')\}$$

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \max_{a' \in \mathcal{A}} Q^*(s', a')$$

2 Model-based Learning

动态规划具有最优子结构。多步的贪心可以收敛到全局极值点。策略迭代 = 策略评估 + 策略提升, 其最优性由 **策略提升定理** 来保证: 贪心的改进, $\pi \rightarrow \pi^*$, 使用 $\pi'(s) = \operatorname{argmax}_{a \in \mathcal{A}} Q^\pi(s, a)$ 可以保证 $V^\pi(s) \leq V^{\pi^*}(s)$, 详细证明见后。

这里总结两个算法:

Policy Iteration

- **Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$**
 1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
 2. Policy Evaluation
 Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s', r} p(s', r|s, \pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
 3. Policy Improvement
 $policy_stable \leftarrow true$
 For each $s \in \mathcal{S}$:
 $old_action \leftarrow \pi(s)$
 $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s', r} p(s', r|s, a) [r + \gamma V(s')]$
 If $old_action \neq \pi(s)$, then $policy_stable \leftarrow false$
 If $policy_stable$, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Value Iteration

- **Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$**
 1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
 2. Policy Evaluation
Loop:
 $\Delta \leftarrow 0$
Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
 3. Policy Improvement
 $policy_stable \leftarrow true$
For each $s \in \mathcal{S}$:
 $old_action \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$
If $old_action \neq \pi(s)$, then $policy_stable \leftarrow false$
If $policy_stable$, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

3 Model-free learning

MC-Value Iteration

- **First-visit** MC control (for ϵ -greedy policy), estimates $\pi \approx \pi_*$
- Initialize:
 - $\pi \leftarrow$ an ϵ -soft policy, value $Q(s, a)$, empty list $Returns(s, a)$, $\forall s, a$
- Repeat (for **each episode**):
 - Sample an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$
 - $G \leftarrow 0$
 - Repeat (for **each step** of episode, $t = T - 1, T - 2, \dots, 0$):
 - $G \leftarrow \gamma G + R_{t+1}$
 - Unless the pair S_t, A_t **appears** in $S_0, A_0, \dots, S_{t-1}, A_{t-1}$:
 - Append G to $Returns(S_t, A_t)$
 - $Q(S_t, A_t) \leftarrow \text{Avg}[Returns(S_t, A_t)]$
 - $A^* \leftarrow \arg\max_a Q(S_t, a)$
 - For all $a \in \mathcal{A}$: $\pi(a|S_t) = \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}| & \text{if } a = A^* \\ \epsilon/|\mathcal{A}| & \text{if } a \neq A^* \end{cases}$

Generalized policy iteration (GPI)

通过蒙特卡洛，我们采样的实际上是真实的分布中的多条完整的trajectory（回报序列），用以估计当前的价值函数，蒙特卡洛算法是对真实的价值函数的无偏估计但是由于每次采样的是不同的时间步，回报的波动会影响每个状态值的更新，方差较大

TD

时序差分算法相当于向未来看了 n 步，采用这 n 步作为更新价值的奖励信号，当 $n \rightarrow \infty$ 实际上呢就得到了蒙特卡洛算法。 $TD(0)$ 只往前看了一步，是有偏的估计，但方差比较小。

TD(0)

- **TD(0)** prediction, for estimating $V \approx v_\pi$
- **Input:** the policy π to be evaluated, step size $\alpha \in [0,1]$
- **Initialize:**
 - Value $V(s), \forall s \in \mathcal{S}$, except that $V(\text{terminal}) = 0$
- **Repeat** (for **each episode**):
 - Initialize S
 - **Repeat** (for **each step** of episode):
 - $A \leftarrow$ action given by π for S
 - Take action A , observe R, S'
 - $V(S) \leftarrow V(S) + \alpha(R + \gamma V(S') - V(S))$
 - $S \leftarrow S'$
 - **Until** S is terminal
- **N-step TD** prediction, for estimating $V \approx v_\pi$
- **Initialize:** Value $V(s), \forall s \in \mathcal{S}$
- **Repeat** (for **each episode**):
 - Initialize and store $S_0 \neq \text{terminal}, T \leftarrow \infty$ (big cost)
 - **Repeat** (for $t = 0, 1, 2, \dots$):
 - If $t < T$:
 - Take an action according to $\pi(\cdot | S_t)$
 - Observe and store next reward R_{t+1} & next state S_{t+1}
 - If S_{t+1} is terminal, then $T \leftarrow t + 1$
 - $\tau \leftarrow t - n + 1$ (τ is the time with state value updated)
 - If $\tau \geq 0$:
 - $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
 - If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$
 - $V(S_\tau) \leftarrow V(S_\tau) + \alpha(G - V(S_\tau))$
 - **Until** $\tau = T - 1$

TD(λ)

为结合MC和TD两种算法的优势，同时提升 $TD(n)$ 算法的**efficiency**，我们引入资格迹(*Eligibility Trace*)的概念，用来度量某个状态对于奖励信号的提示强度(因为奖励不是及时的，我们需要衡量对**trajectory**中的奖励信号进行一个猜测)。可以理解为对状态发生频率以及最终的奖励发生前的状态两种**heuristics**进行一个合理的tradeoff，我们让

$E_0(s) = 0 \quad E_t(s) = \lambda \gamma E_{t-1}(s) + \mathbb{1}(S_t = s)$. 基于此可以得到 $TD(\lambda)$ 的算法:

- Input: the policy π , discount γ , decay rate λ .

- Repeat (for **each episode**)

- $E(s) \leftarrow 0$ for all $s \in \mathcal{S}$

- Initialize S

- Repeat (for **each step** of episode)

- Choose $A \sim \pi(\cdot | S)$

- Take action A , observe R, S'

- TD error: $\delta = R + \gamma V(S') - V(S)$

- Eligibility trace: $E(S) \leftarrow E(S) + 1$

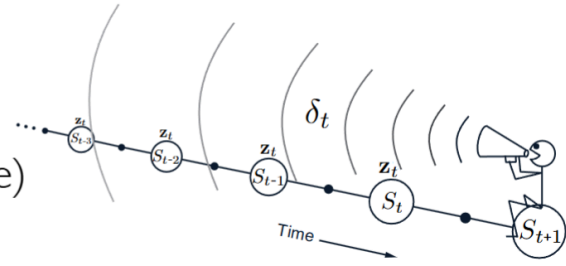
- For all $s \in \mathcal{S}$

- $V(s) \leftarrow V(s) + \alpha \delta E(s)$

- Eligibility trace: $E(s) = \gamma \lambda E(s)$

- $S \leftarrow S'$

- Until S is terminal



$z_t = E$ at step t

SARSA

我们希望对动作价值函数 $Q(S, A)$ 给出一个好的估计，从而产生基于TD的控制算法。

SARSA

- Initialize: $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ arbitrarily, $Q(\text{terminal}, \cdot) = 0$

- Repeat (for **each episode**):

- Initialize S

- Choose A from S using policy derived from Q (e.g. **ϵ -greedy**)

- Repeat (for **each step** of episode):

- Take action A , observe R, S'

- Choose A' from S' using policy derived from Q (e.g. **ϵ -greedy**)

- $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

- $S \leftarrow S', A \leftarrow A'$

- Until S is terminal

On-policy

SARSA(λ)

- Initialize: $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
- Repeat (for each episode):
 - $E(s, a) \leftarrow 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
 - Initialize S, A
 - Repeat (for each step of episode):
 - Take action A , observe R, S'
 - Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)
 - TD error: $\delta \leftarrow R + \gamma Q(S', A') - Q(S, A)$
 - Eligibility trace: $E(S, A) \leftarrow E(S, A) + 1$
 - For all $s \in \mathcal{S}, a \in \mathcal{A}(s)$:
 - $Q(s, a) \leftarrow Q(s, a) + \alpha \delta E(s, a)$
 - Eligibility trace: $E(s, a) \leftarrow \gamma \lambda E(s, a)$
 - $S \leftarrow S', A \leftarrow A'$
 - Until S is terminal

Q-learning

我们希望采用探索与利用平衡的策略来收集数据，但使用贪婪的方式进行控制，于是我们可以得到如下的离线学习算法，即著名的Q-learning算法。

- Initialize: $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ arbitrarily, $Q(\text{terminal}, \cdot) = 0$
- Repeat (for each episode):
 - Initialize S
 - Repeat (for each step of episode):
 - Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 - Take action A , observe R, S'
 - $Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_a Q(S', a) - Q(S, A) \right]$
 - $S \leftarrow S'$
 - Until S is terminal